Gustavo Alves Vieira
Daniel Carvalho de Araujo

Análise de Impacto no Tempo de Renderização de um Ray Tracer Utilizando as Estruturas de Particionamento Espacial BVH e KD-Tree

São Paulo
2026

Gustavo Alves Vieira
Daniel Carvalho de Araujo

Análise de Impacto no Tempo de Renderização de um Ray Tracer Utilizando as Estruturas de Particionamento Espacial BVH e KD-Tree

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do curso de Ciencia da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Ciencia da computação

Orientador: Prof. Me. Leonardo Massayuki Takuno

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é realizar a comparação e análise do tempo de renderização de um Ray Trace utilizando estruturas de particionamento BVH e KD-Tree. A justificativa para este estudo baseia-se em medir a perfomance computacional ao escolher uma das estruturas de particionamento apresentadas, isso se faz necessário para descobrir a abordagem com menor custo computacional. A revisão bibliográfica abrange a analise do impacto no tempo de renderização ao escolher uma tecnica especifica de particionamento espacial. Como solução proposta, pretende-se implementar uma pequna pipeline de renderização utilizando-se ambas as tecnicas de particionamento e então comparar o impacto no tempo de processamento. Este trabalho está organizado em três capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: ray tracing. computação grafica. tracejo de raio.

Sumário

1 INTRODUÇÃO 4
1.1 Contexto 4
1.2 Justificativa 5
1.3 Objetivo 6
1.3.1 Objetivo principal 6
1.3.2 Objetivos secundários 7

2 REVISÃO BIBLIOGRÁFICA 8
2.1 Referencial teórico 8
2.2 Metodologia da pesquisa bibliográfica 8
2.3 Fundamentos 9
2.4 Trabalhos relacionados 9
2.4.1 Trabalho 1: [Título do trabalho] 10
2.4.2 Trabalho 2: [Título do trabalho] 10
2.4.3 Trabalho 3: [Título do trabalho] 10

3 DESENVOLVIMENTO 11
3.1 Solução proposta 11
3.2 Cronograma de trabalho 11

1 Introdução

Simular fielmente cenários reais em telas 2D utilizando computadores é um desafio que acompanha a computação gráfica desde o início de seu estudo. Adaptamos conceitos visuais observados no mundo real e realizamos pequenos ajustes para que se torne possível de ter um resultado satisfatório em telas de computador com o poder computacional limitado que temos (ANDREW S. GLASSNER, 1998).

O traçado de raios (ray tracing) é responsável por ajudar no sombreamento, reflexos e iluminação da cena e em calcular como os objetos renderizados em cena se comportam diante de uma fonte de luz externa, esse técnica é uma "imitação"de como o olho humano processa particulas de luz, simular essa ocorrencia de luz nos computadores não é uma tarefa fácil, se quisessem calcular a incidência da uma fonte de luz sobre N objetos teríamos que disparar milhões de raios para todos os objetos da cena e calcular o quanto essa fonte de luz está afetando aquele determinado pixel.

Ao invés de calcular milhões de raios disparados, disparamos um quantidade menor de raios direto de um "olho"aritificial (uma camera por exemplo), e então a partir de onde esses raios atingiram na nossa cena realizamos o calculo para todas as fontes de luza disponiveis.

Mesmo realizando essa inversão, o calculo de raios ainda demanda um pode computacional extramamente alto dependendo do quão fiel desejamos que a imagem gerada artificialmente seja. Analisar e estudar tecnicas como o BVH e KD-Tree nos ajudam a viabilizar cada vez mais imagems de maior qualidade com hardware limitado.

1.1 Contexto

A síntese de imagens em Computação Gráfica tem como um de seus maiores desafios a criação de uma imagem bidimensional que represente com realismo um mundo tridimensional. Nesse cenário, o ray tracing (traçado de raios) consolidou-se como uma técnica essencial devido à sua capacidade de simular fielmente o comportamento físico da luz, rastreando fenômenos complexos como reflexão, refração e sombras através de fótons virtuais. Para viabilizar essa simulação, o algoritmo padrão opera de forma reversa (backward ray tracing), traçando raios que partem do olho do observador (câmera), passam pelos pixels da tela e viajam em direção aos objetos da cena.Apesar do elevado nível de fotorrealismo proporcionado, o ray tracing esbarra em um problema estrutural crônico: o seu alto custo computacional. A base do algoritmo exige que se pergunte repetidamente "qual objeto este raio atinge?". Como demonstrado em literaturas fundamentais da área, como o artigo Ray Tracing in One Weekend, a abordagem ingênua que calcula a interseção de um raio contra uma lista não estruturada contendo todos os objetos geométricos da cena

é extremamente custosa e impraticável para cenários de média a alta densidade.Indicadores, Lacunas e o Domínio do ProblemaÉ neste domínio — a otimização da travessia de raios e a redução geométrica do ambiente virtual — que a ineficiência do modelo básico se manifesta. O gargalo se intensifica porque o custo computacional e o número de testes de interseção crescem drasticamente conforme a complexidade da cena e o número de primitivas geométricas aumentam.

Para suprir essa lacuna e viabilizar a técnica, a área de estudo adotou estruturas de aceleração baseadas em particionamento espacial, sendo o BVH (Bounding Volume Hierarchy) e a KD-Tree duas das abordagens mais proeminentes. A lacuna de pesquisa atual reside na necessidade de análises empíricas e controladas que avaliem como diferentes cenários de complexidade geométrica, o tempo gasto na construção dessas estruturas (pré-processamento) e a profundidade das árvores influenciam, na prática, o custo computacional geral e a redução do tempo total de renderização.Afetados, Consequências e RelevânciaOs impactos desse gargalo de desempenho não se restringem à academia. Os principais afetados são os setores industriais que dependem diretamente de ambientes virtuais de alta fidelidade, como estúdios de cinema, empresas de efeitos visuais (VFX), desenvolvedores de jogos eletrônicos, projetistas arquitetônicos e pesquisadores de simulações médicas.

A consequência de ignorar ou não mitigar esse problema é severa: sem estruturas de aceleração altamente otimizadas, a interatividade e a geração de gráficos fotorrealistas em tempo real tornam-se virtualmente impossíveis. Além disso, o alto processamento e consumo de recursos de memória (CPU/RAM) resultam em custos operacionais e energéticos exorbitantes com gigantescas fazendas de renderização (render farms). Dessa forma, a relevância desta pesquisa empírica para a Computação e para a sociedade é indiscutível. Ao comparar o comportamento computacional do BVH e da KD-Tree no controle do fluxo de interseções de raios com objetos , o estudo propiciará um embasamento prático essencial para desenvolvedores de motores gráficos na tomada de decisão arquitetural. Otimizar algoritmos de síntese de imagens colabora diretamente para processos industriais mais rápidos e para a democratização da computação visual de alto nível.

1.2 Justificativa

Este trabalho contribui para a literatura de Computação Gráfica e Estruturas de Dados ao fornecer uma avaliação empírica e rigorosa sobre a eficiência algorítmica no traçado de raios. Embora a base matemática e óptica do ray tracing seja amplamente conhecida , a compreensão profunda de como diferentes métodos de particionamento espacial otimizam a travessia de raios permite avanços no estudo da complexidade de algoritmos. Ao implementar e mensurar o comportamento do BVH e da KD-Tree em C++ , a pesquisa gera dados concretos sobre a escalabilidade geométrica, fomentando discussões acadêmicas sobre o balanço entre o custo de construção de estruturas de dados e o tempo

de execução.

Em aplicações industriais — como produção de efeitos visuais (VFX), desenvolvimento de jogos eletrônicos, simulações físicas e visualização arquitetônica —, o tempo de renderização traduz-se diretamente em custos de infraestrutura e consumo de recursos (CPU/RAM). Ao comparar o desempenho prático de estruturas de aceleração , este estudo entregará diretrizes claras para desenvolvedores de software e engenheiros gráficos na tomada de decisão sobre qual técnica empregar. A redução da quantidade de testes de interseção resulta em render farms mais eficientes e em fluxos de trabalho mais ágeis para artistas digitais, que dependem de iterações rápidas para avaliar a iluminação e os materiais de suas cenas.

Apesar de guias fundamentais da área (como a série Ray Tracing in One Weekend) e livros clássicos introduzirem a importância das estruturas de aceleração para evitar a complexidade linear da força bruta, existe uma carência de estudos recentes focados na comparação direta, controlada e isolada dos trade-offs entre o BVH e a KD-Tree. Grande parte dos projetos foca na implementação de novos efeitos visuais (física de superfícies, motion blur, etc.), assumindo a estrutura de dados como uma "caixa preta". O que ainda não foi suficientemente explorado na prática é a investigação pormenorizada de como variáveis isoladas — como o número de objetos, a profundidade das árvores topológicas e o tempo gasto na construção prévia das estruturas — afetam o tempo de renderização em diferentes topologias de cenas tridimensionais. Este TCC preenche essa lacuna ao isolar o algoritmo e monitorar detalhadamente as métricas de desempenho sob essas condições

1.3 Objetivo

1.3.1 Objetivo principal

Analisar experimentalmente o impacto das estruturas de particionamento espacial BVH e KD-Tree no tempo de renderização de um ray tracer, por meio do desenvolvimento e da comparação de implementações.

O objetivo principal deve ser **único** e sintetizar o propósito central do trabalho em uma frase direta. Ele deve responder à pergunta: *o que este trabalho pretende resolver ou alcançar?*

Exemplo: *Desenvolver um sistema web para gerenciamento de tarefas acadêmicas que permita aos estudantes organizar suas atividades de forma colaborativa.*

Dica: utilize verbos no infinitivo como *desenvolver*, *analisar*, *propor*, *avaliar*, *implementar*, *comparar*, entre outros. Evite verbos vagos como *estudar* ou *entender*.

1.3.2 Objetivos secundários

Os objetivos secundários são desdobramentos do objetivo principal. Eles descrevem as etapas ou ações necessárias para atingir o objetivo principal. Devem ser mensuráveis e verificáveis.

- Desenvolver um ray tracer básico em C++, capaz de renderizar cenas tridimensionais contendo diferentes quantidades de objetos geométricos.
- Implementar estruturas de aceleração baseadas em BVH (Bounding Volume Hierarchy) e KD-Tree, integrando-as ao processo de interseção de raios com objetos da cena.
- Comparar o desempenho das duas estruturas de particionamento espacial, analisando a redução no número de testes de interseção entre raios e primitivas geométricas.
- Avaliar o impacto de cada estrutura no tempo total de renderização, considerando diferentes níveis de complexidade das cenas.
- Analisar o custo de construção das estruturas BVH e KD-Tree e seu impacto no tempo total do processo de renderização.
- Investigar como fatores como número de objetos, profundidade da árvore e organização espacial da cena influenciam o desempenho das estruturas.

2 Revisão Bibliográfica

A revisão bibliográfica tem como finalidade apresentar o embasamento teórico do trabalho, demonstrando o conhecimento do autor sobre o tema e identificando o estado da arte. Este capítulo deve ser organizado por conceitos e temas, construindo uma linha de argumentação que sustente a proposta do trabalho.

2.1 Referencial teórico

Apresente os principais conceitos, teorias e definições que fundamentam o trabalho. O referencial teórico deve fornecer ao leitor a base necessária para compreender o problema e a solução proposta.

Organize esta seção por temas ou conceitos, não por autor. Cada subseção deve abordar um conceito relevante, explicando-o e relacionando-o ao contexto do trabalho.

Por exemplo: se o trabalho envolve desenvolvimento web, aborde conceitos como arquitetura cliente-servidor, APIs REST, padrões de projeto, etc. Se envolve inteligência artificial, explique os algoritmos e técnicas que serão utilizados.

Utilize citações para fundamentar cada conceito apresentado. Segundo **??**), o referencial teórico deve demonstrar que o autor domina os fundamentos necessários para conduzir a pesquisa.

2.2 Metodologia da pesquisa bibliográfica

Descreva como a pesquisa bibliográfica foi conduzida, incluindo:

- **Bases de dados consultadas**: Google Scholar, IEEE Xplore, ACM Digital Library, SciELO, Periódicos CAPES, entre outras;
- **Palavras-chave utilizadas**: liste os termos de busca em português e inglês;
- **Critérios de inclusão e exclusão**: quais critérios foram usados para selecionar ou descartar trabalhos (período, idioma, relevância, tipo de publicação);
- **Período de abrangência**: intervalo de anos das publicações consultadas.

Exemplo de descrição:

A pesquisa bibliográfica foi realizada nas bases Google Scholar, IEEE Xplore e ACM Digital Library, utilizando as palavras-chave "machine learning", "classificação de texto" e "processamento de linguagem natural". Foram selecionados artigos publicados entre

2018 e 2025, em português e inglês, que abordassem diretamente técnicas de classificação automática de documentos.

2.3 Fundamentos

Aprofunde os conceitos técnicos que serão utilizados na solução proposta. Diferentemente do referencial teórico (que apresenta conceitos gerais), os fundamentos devem detalhar as tecnologias, algoritmos, frameworks ou metodologias específicas que serão empregados no desenvolvimento.

Por exemplo:

- Se o trabalho utiliza aprendizado de máquina, explique os algoritmos escolhidos (árvores de decisão, redes neurais, SVM, etc.);
- Se o trabalho envolve desenvolvimento de software, descreva as tecnologias (linguagens, frameworks, bancos de dados);
- Se envolve análise de dados, explique as métricas e técnicas estatísticas que serão aplicadas.

Utilize figuras, diagramas e tabelas para ilustrar conceitos quando apropriado.

2.4 Trabalhos relacionados

Apresente até **3 trabalhos** que **compartilhem o mesmo foco de resolução de problema** que o seu. O critério principal de seleção é: os trabalhos escolhidos devem ter tentado resolver o **mesmo problema** (ou um problema muito próximo) ao que você está propondo resolver. Não basta que o trabalho aborde o mesmo tema ou utilize a mesma tecnologia — ele deve ter enfrentado o mesmo desafio prático ou teórico.

Por que esse critério é importante? Ao selecionar trabalhos que atacam o mesmo problema, você demonstra que:

- O problema é relevante e já foi reconhecido por outros pesquisadores;
- Existem soluções anteriores, mas com limitações que o seu trabalho pretende superar;
- Seu trabalho não é uma repetição, mas um avanço em relação ao que já foi feito.

Para cada trabalho relacionado, descreva:

1. **Qual problema** o trabalho tentou resolver (e por que é o mesmo problema que o seu);
2. A metodologia ou abordagem utilizada para resolvê-lo;

3. Os principais resultados obtidos;
4. As limitações identificadas na solução proposta;
5. Como o **seu trabalho se diferencia** ou avança em relação a ele.

2.4.1 Trabalho 1: [Título do trabalho]

??) abordou o problema de [descrever o problema — deve ser o mesmo ou muito similar ao seu]. Para resolvê-lo, os autores propuseram [descrever a solução]. A metodologia utilizada foi [descrever]. Os resultados mostraram que [descrever]. No entanto, a solução apresenta limitações como [descrever as lacunas que permanecem]. O presente trabalho diferencia-se por [explicar como sua proposta avança na resolução do mesmo problema].

2.4.2 Trabalho 2: [Título do trabalho]

[Descrever o segundo trabalho seguindo a mesma estrutura acima, sempre evidenciando que o problema abordado é o mesmo ou muito próximo ao seu.]

2.4.3 Trabalho 3: [Título do trabalho]

[Descrever o terceiro trabalho seguindo a mesma estrutura acima, sempre evidenciando que o problema abordado é o mesmo ou muito próximo ao seu.]

Uma tabela comparativa ajuda a sintetizar as diferenças entre os trabalhos:

Tabela 1 – Comparação entre trabalhos relacionados

Critério	**Trabalho 1**	**Trabalho 2**	**Trabalho 3**	**Este trabalho**
Objetivo	[resumo]	[resumo]	[resumo]	[resumo]
Tecnologia	[tecn.]	[tecn.]	[tecn.]	[tecn.]
Validação	[método]	[método]	[método]	[método]
Limitação	[limit.]	[limit.]	[limit.]	—

Fonte: Elaborado pelo autor (2025)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

Descreva detalhadamente a solução que será desenvolvida para resolver o problema apresentado na introdução. A descrição deve incluir:

- **Visão geral da solução**: o que será desenvolvido (sistema, aplicativo, modelo, ferramenta, etc.);
- **Arquitetura**: como os componentes da solução se organizam e interagem;
- **Tecnologias e ferramentas**: quais linguagens, frameworks, bibliotecas, bancos de dados e ambientes serão utilizados, e por que foram escolhidos;
- **Funcionalidades principais**: o que a solução fará para atender aos objetivos;
- **Metodologia de desenvolvimento**: qual abordagem será utilizada (ágil, cascata, prototipação, etc.);
- **Critérios de validação**: como será medido o sucesso da solução (métricas, testes, avaliação com usuários, etc.).

Utilize diagramas, fluxogramas ou mockups para ilustrar a solução. Exemplo de descrição de arquitetura:

A solução proposta consiste em uma aplicação web composta por três camadas: front-end desenvolvido em React.js, back-end em Node.js com Express, e banco de dados PostgreSQL. A comunicação entre front-end e back-end será feita por meio de uma API REST.

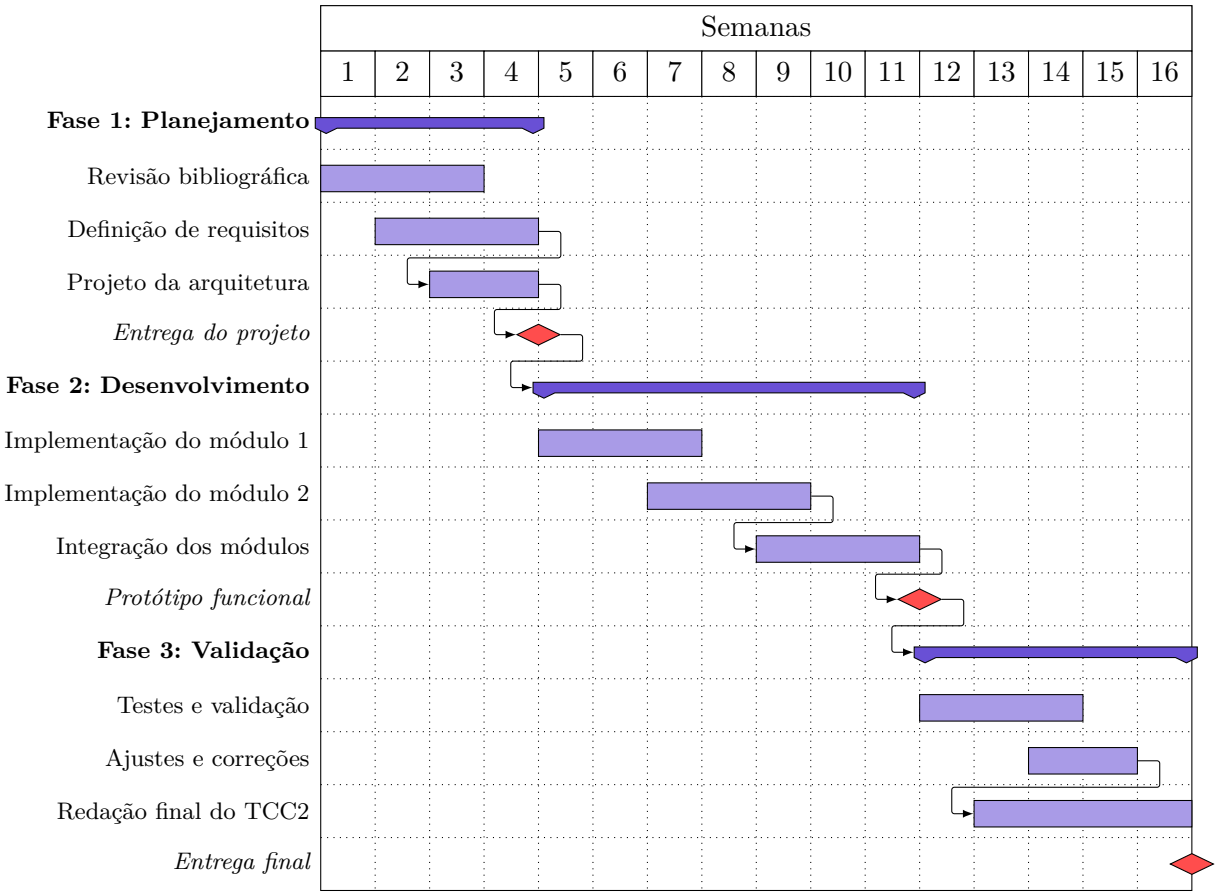
3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Instruções para personalização do cronograma:

- Ajuste o número de semanas conforme o calendário acadêmico do semestre;
- Modifique as atividades de acordo com as etapas do seu projeto;

Figura 1 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2025)

- Os marcos (*milestones*) indicam entregas importantes — adapte-os às datas de avaliação;
- As setas entre atividades indicam dependências — uma atividade só inicia após a conclusão da anterior;
- Utilize o diagrama como ferramenta de acompanhamento: atualize-o a cada reunião com o orientador.